

Full Length Article

Quantification via gaussian latent space representations

Olaya Pérez-Mon [✉], Juan José del Coz [✉], Pablo González [✉]

Artificial Intelligence Center, University of Oviedo, Gijón, 33204, Asturias, Spain

ARTICLE INFO

Keywords:

Quantification
Prevalence estimation
Deep learning
Machine learning
Neural networks

ABSTRACT

Quantification, or prevalence estimation, is the task of predicting the prevalence of each class within an unknown bag of examples. Most existing quantification methods in the literature rely on prior probability shift assumptions to create a quantification model that uses the predictions of an underlying classifier to make optimal prevalence estimates. In this work, we present an end-to-end neural network that uses Gaussian distributions in latent spaces to obtain invariant representations of bags of examples. This approach addresses the quantification problem using deep learning, enabling the optimization of specific loss functions relevant to the problem and avoiding the need for an intermediate classifier, tackling the quantification problem as a direct optimization problem. Our method achieves state-of-the-art results, both against traditional quantification methods and other deep learning approaches for quantification. The code needed to reproduce all our experiments is publicly available at <https://github.com/AICGijon/gmnet>.

1. Introduction

Across a wide range of machine learning tasks, deep learning has proven to be a very useful, even a revolutionary tool to obtain amazing results in a wide variety of applications. These include generative AI (text, image, video), translation, or computer vision, among others. Despite its unprecedented success in these areas, the application of deep learning techniques in more traditional machine learning tasks, such as quantification, has been less straightforward.

Quantification, or prevalence estimation, involves the challenge of developing models capable of accurately estimating the prevalence of different classes within sets of examples unseen during training. These sets, often referred to as ‘bags’, are affected by some kind of dataset shift (typically prior probability shift), which makes the problem non-trivial. In the field of quantification, there is a variety of machine learning methods designed to tackle this problem (Esuli et al., 2023; González et al., 2017). Among these methods, the most successful ones (which will be introduced in the following section) typically employ an underlying classifier as the foundation upon which the quantification method is built (Schumacher et al., 2021).

A key aspect of these quantification methods is their learning assumption. Under prior probability shift assumptions (Moreno-Torres et al., 2012), the distribution of data varies between train and test, characterized by the condition $P_{tr}(\mathcal{Y}) \neq P_{tst}(\mathcal{Y})$ while maintaining $P_{tr}(\mathcal{X}|\mathcal{Y}) = P_{tst}(\mathcal{X}|\mathcal{Y})$. In other words, the prevalence of the classes changes between train and test, but the class conditionals remain constant. In particu-

lar, the assumption $P_{tr}(\mathcal{Y}) \neq P_{tst}(\mathcal{Y})$ makes that the obvious approach to solving quantification, that is 1) train a classifier using training data; 2) classify the examples in the test bag; and 3) count how many belong to each class does not work in practice. This method is named Classify & Count (CC) in the quantification literature and is not suited for quantification problems because the classifier’s output probabilities are calibrated based on the training distribution. When the class distribution changes in the test bags, these probabilities no longer reflect the true prevalence of classes, leading to biased and inaccurate prevalence estimates. Specifically, this discrepancy may cause the method to overestimate/underestimate the prevalence of a given class when its true prevalence decreases/increases compared to that observed in the training data.

Recent advancements in quantification have introduced deep learning methods that diverge from classical approaches (Esuli et al., 2018; Pérez-Mon et al., 2024; Qi et al., 2021). Unlike traditional methods, some of these deep learning techniques operate without explicit learning assumptions and can directly utilize bags of examples without the need for individual example annotations. Another distinguishing feature is their capability to optimize a user-defined loss function, tailored to the specific requirements of the quantification tasks. A fundamental characteristic of deep learning techniques applied to quantification is their focus on deriving a representative embedding for each bag of examples, crucial for estimating the bag class prevalences (the final objective). This bag representation serves as a compressed representation of the entire bag and should have the necessary information to facilitate

* Corresponding author.

E-mail addresses: perezolaya@uniovi.es (O. Pérez-Mon), juanjo@uniovi.es (J.J. del Coz), gonzalezpablo@uniovi.es (P. González).

<https://doi.org/10.1016/j.neunet.2026.108886>

Received 22 January 2025; Received in revised form 24 January 2026; Accepted 21 March 2026

Available online 22 March 2026

0893-6080/© 2026 The Authors. Published by Elsevier Ltd. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

the reconstruction of class prevalences present within the bag. The use of bags annotated by prevalence directly as input data transforms the learning task from an *asymmetric* one to a *symmetric* one (Pérez-Mon et al., 2024). In the traditional *asymmetric* approach, a model is trained on individually labeled examples to predict bag prevalences, typically relying on an intermediate classifier. By contrast, the *symmetric* approach addresses the quantification task more directly, using bags annotated by prevalence as input. This eliminates the need for classification as an intermediate step, allowing the model to focus directly on the quantification problem.

In this work, we propose a new layer for representing bags of examples, based on modeling latent spaces using Gaussian distributions. The idea is to improve previous representation layers, such as pooling layers (Qi et al., 2021) or histograms (Pérez-Mon et al., 2024). We prove through experimentation that this bag representation is more powerful than the previous ones, obtaining unprecedented results in public datasets included in the only two quantification competitions held so far. Apart from the layer's inner workings and its parameter initialization, we also discuss a regularization technique, specific to this type of layer, that may help improve the results and the convergence time. Finally, we perform extensive experimentation using two distinct data modalities. First, we evaluate the method on three tabular product-review datasets from the LeQua quantification competitions. Second, to demonstrate the modality-agnostic benefits of our approach, testing it against the CIFAR-10 and CIFAR-100 visual benchmarks. This dual evaluation allows us to study the advantages and limitations of both traditional and deep learning quantifiers across structured text and image datasets.

The rest of the paper is organized as follows. Section 2 introduces classical learning methods in the quantification literature. Then, in Section 3, we discuss previous approaches using deep learning for tackling the quantification problem, proposing our own layer for representing bags, discussing parameter initialization, regularization, and the use of example labels if available. Section 4 details our experimental setup and findings. Finally, Section 5 presents our conclusions and outlines avenues for future research.

2. Related work: Previous work on quantification

In this section, we are going to briefly present the most commonly used quantification methods in the literature. We refer the reader to Esuli et al. (2023), González et al. (2017) for a more in-depth explanation of these methods.

Given a labeled training set $D_{tr} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, drawn from a distribution $P_{tr}(\mathcal{X}, \mathcal{Y})$ where $\mathcal{X}$ represents the feature space and $\mathcal{Y}$ the set of true class labels $\{c_1, c_2, \dots, c_l\}$, and an unlabeled test bag $B = \{\mathbf{x}_j\}_{j=1}^m$ drawn from a different distribution $P_{tst}(\mathcal{X}, \mathcal{Y})$ where $P_{tr}(\mathcal{Y}) \neq P_{tst}(\mathcal{Y})$, the goal is to estimate the prevalence (or proportion) $\hat{P}(\mathcal{Y})$ of each class in the test bag, assuming that $P_{tr}(\mathcal{X}|\mathcal{Y}) = P_{tst}(\mathcal{X}|\mathcal{Y})$.

2.1. Quantification Methods

One of the first methods in the quantification literature is the Adjusted Classify and Count (ACC) method (see Forman (2008)), also known as Black Box Shift Estimation (BBSE) (Lipton et al., 2018). It learns a classifier $h: \mathcal{X} \rightarrow \mathcal{Y}$ from training data and then applies a correction. First, we compute the probability of predicting that a random example $x \in B$ is classified as class c_i :

$$p(h(\mathbf{x}) = c_i) = \sum_{c_j \in \mathcal{Y}} p(h(\mathbf{x}) = c_i | y = c_j) \cdot p(c_j), \quad (1)$$

where $p(h(\mathbf{x}) = c_i | y = c_j)$ is the probability that h predicts c_i when the actual class of x is c_j and $p(c_j)$ is the true prevalence of class c_j in the test bag. The former probability can be estimated using many-fold cross-validation over the train set, while the latter is the quantity that we try to find out. Writing this same equation for all the classes, we get a system of l equations that we can solve obtaining the values

$[p(c_1), p(c_2), \dots, p(c_l)]$ which will be the model predictions for the bag class prevalences $[\hat{p}_1, \hat{p}_2, \dots, \hat{p}_l]$.

In Bella et al. (2010), the authors propose probabilistic versions of the methods Classify and Count (CC) and Adjusted Classify and Count (ACC). Both methods replace the hard classifier h with a soft classifier $s: \mathcal{X} \rightarrow [0, 1]^l$. The former, named Probabilistic Classify and Count (PCC), just replaces basic class counting by averaging the probabilistic outputs of the classifier by class:

$$\hat{p}_l = \frac{1}{m} \sum_{\mathbf{x}_j \in B} s(\mathbf{x}_j)[c_l]. \quad (2)$$

On the other hand, ACC is transformed into Probabilistic Adjusted Classify and Count (PACC), and solves a system equivalent to the one used in ACC but based on the computation of posterior probabilities averaged over D_{tr} and B .

Another family of quantification methods is based on matching a mixture of the training distribution and the testing distribution (Malet-zke et al., 2019), based on estimates from D_{tr} and B respectively. The training distribution mixture D'_{tr} is modeled as a combination of class-specific distributions of each class c_j , $D_{tr}^{c_j}$, weighted by their respective estimated prevalence $\hat{p}_j$:

$$D'_{tr} = \sum_{c_j \in \mathcal{Y}} D_{tr}^{c_j} \cdot \hat{p}_j. \quad (3)$$

The idea is to minimize a distance, Δ , between D'_{tr} and the test bag distribution B' :

$$\argmin_{\hat{p}_1, \dots, \hat{p}_l} \Delta(D'_{tr}, B') = \argmin_{\hat{p}_1, \dots, \hat{p}_l} \Delta\left(\sum_{c_j \in \mathcal{Y}} D_{tr}^{c_j} \cdot \hat{p}_j, B'\right). \quad (4)$$

Quantification algorithms based on matching distributions consist of three different elements: 1) a method to estimate data distributions, 2) a similarity measure Δ , and 3) an optimization method to solve Eq. 4 (Castaño et al., 2023). One popular choice is training a probabilistic classifier s and representing the distributions using histograms over the example predictions (González-Castro et al., 2013). In the experiments, for the similarity measure Δ , we have selected the Hellinger distance (González-Castro et al., 2013). We refer to this method as DMy in the results section.

The Expectation Maximization for Quantification (EMQ) method (Saerens et al., 2002) uses the EM algorithm to adjust the posterior probabilities produced by a soft classifier s in response to potential shifts in label distribution. This process involves iterating through mutually recursive steps: expectation (updating the posteriors) and maximization (updating the priors) until convergence is achieved. This method is among the most competitive in quantification literature (Esuli et al., 2022), but it heavily relies on the calibration of the posterior probabilities returned by the soft classifier s (Alexandari et al., 2020).

2.2. Measuring quantification error

To evaluate the performance of a quantification model, it is necessary to measure the discrepancy between the true prevalence vector p and the estimated prevalence vector $\hat{p}$ for a given bag of examples. While various metrics exist in the literature for this purpose (see Sebastiani (2020)), in this work we focus on two specific error measures that are standard for the multiclass and ordinal tasks addressed in our experiments.

Relative Absolute Error (RAE) is a commonly used metric for multiclass quantification tasks. It is defined as:

$$\text{RAE}(\mathbf{p}, \hat{\mathbf{p}}) = \frac{1}{|\mathcal{Y}|} \sum_{c_i \in \mathcal{Y}} \frac{|\delta(p(c_i)) - \delta(\hat{p}(c_i))|}{\delta(p(c_i))}, \quad (5)$$

in which $\delta(p_i) = \frac{p_i + \epsilon}{|\mathcal{Y}| + 1}$ is the smoothing function, with ϵ the smoothing factor that is set to $2|B| - 1$ following Sebastiani (2020).

Normalized Matched Distance (NMD) is employed for ordinal quantification tasks, such as sentiment polarity estimation, where classes have an implicit order. NMD is defined as:

$$\text{NMD}(\mathbf{p}, \hat{\mathbf{p}}) = \frac{1}{|\mathcal{Y}| - 1} \sum_{j=1}^{|\mathcal{Y}|-1} \left| \sum_{i=1}^j \hat{p}(c_i) - \sum_{i=1}^j p(c_i) \right|, \quad (6)$$

where $\frac{1}{|\mathcal{Y}|-1}$ is a normalization term so NMD ranges between 0 (best) and 1 (worst). NMD is designed to guide the optimization process in ordinal quantification tasks by quantifying how well the predicted prevalences match the true prevalences in a normalized and class-aware manner, where class order is taken into account.

3. Tackling quantification using deep learning

Quantification was historically tackled with machine learning algorithms that worked under the prior probability shift conditions (see Section 2). Usually, these methods worked using the predictions of a classifier trained over individual examples. Recently, deep learning techniques have allowed to reformulate the task into an end-to-end quantification task (Pérez-Mon et al., 2024).

The difference between the two approaches is notable: traditional methods use an *asymmetric* approach, requiring a labeled training set $D_{\text{tr}} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where each individual example $\mathbf{x}_i$ is associated with a label $y_i \in \mathcal{Y}$. In this approach, a classifier $h : \mathcal{X} \rightarrow \mathcal{Y}$ or $s : \mathcal{X} \rightarrow [0, 1]^I$ is trained on D_{tr} to predict labels or posterior probabilities for individual examples, and the quantifier is derived by aggregating these predictions over a test bag $B = \{\mathbf{x}_j\}_{j=1}^m$, often assuming prior probability shift between training and test data. This approach does not directly minimize a quantification-specific loss and instead relies on classification as an intermediate step.

In contrast, deep learning methods (Pérez-Mon et al., 2024; Qi et al., 2021) adopt a *symmetric* approach, where the model is trained directly on bags of examples $D_{\text{tr}} = \{(B_i, \mathbf{p}_i)\}_{i=1}^n$, with each training bag $B_i = \{\mathbf{x}_j\}_{j=1}^m$ labeled by its prevalence vector $\mathbf{p}_i = [p_i(c_1), \dots, p_i(c_I)]$, where $p_i(c_k)$ is the proportion of class c_k in the bag i . During training, the network weights are optimized to predict the prevalence vector $\mathbf{p}_i$ for each training bag, by directly minimizing a quantification-specific loss, such as the quantification losses defined in Section 2.2. This eliminates the need for intermediate classification and allows the model to focus on learning group-level patterns, providing more accurate and robust prevalence estimates. Notice also that this approach does not assume a particular shift (prior probability shift, covariate shift, etc.); in fact, the training bags may present any shift.

3.1. Previous deep learning approaches used in quantification

In this section, we will summarize previous deep learning approaches developed to address the quantification problem.

The first approach, called QuaNet (Esuli et al., 2018), introduces an innovative method using a Recurrent Neural Network (RNN) for binary sentiment quantification. This involves estimating the relative frequency of positive and negative sentiment labels in a collection of texts. QuaNet utilizes the predictions from an external probabilistic classifier s , which are sorted and fed into the network. This enables the RNN module to learn to identify the switching point between the positive and negative classes. Additionally, prevalence predictions computed using traditional quantifiers like CC, PCC, ACC, and PACC are concatenated in the last layers of the network to enhance the representation obtained by the RNN. Even though QuaNet was the first deep neural network designed to tackle the quantification problem, it has some major drawbacks. First, it needs an underlying classifier, which is required to obtain the traditional quantifiers predictions, which means that the method needs example labels to be trained. Second and more important, it only works for binary quantification, as its concept of a cut point relies on having just two

classes in the dataset. This limitation makes it unsuitable for this study as we are dealing with problems with more than two classes.

The second attempt to solve quantification using deep learning was proposed by Qi et al. (2021) and it is called Deep Quantification Network (DQN). This paper proposes a deep learning architecture to tackle quantification using a *symmetric* approach. The architecture consists of three main components:

1. **Feature Extraction Module (FEM):** This module's layers vary based on the specific data and may include fully connected layers, Convolutional Neural Networks (CNNs), transformers, etc. It is in charge of computing feature vectors for individual examples, applying transformations to the examples, and projecting them to a latent space. Its primary purpose in the pipeline is to project high-dimensional or raw input data into a lower-dimensional latent space that is manageable and optimized for the subsequent Bag Representation Module (BRM).
2. **Bag Representation Module (BRM):** This module summarizes all the feature vectors of the examples in a bag into a single feature vector representing the entire bag. The BRM in DQN utilizes basic pooling layers, such as *average*, *max*, or *median*, which are permutation invariant to the order of the examples within the bag.
3. **Quantification Module (QM):** This module connects the bag representation to a final layer that predicts the prevalence of each class. The QM comprises a set of fully connected layers with a final softmax activation function to ensure that the predicted prevalences sum to one.

It is important to note that the approach proposed by Qi et al. (2021) was previously proposed by Zaheer et al. (2017), just not for quantification purposes but for set processing in general. In their paper, the authors emphasize the importance of a permutation invariant representation when dealing with sets. The underlying intuition is straightforward: the feature vector computed for the set should remain the same regardless of the order of the examples within the set. This characteristic is evidently applicable to the quantification task as well.

The main advantage of the DQN architecture is that it offers a *symmetric* solution to the quantification problem while also being capable of optimizing a specific loss function. However, the primary drawback is that the basic pooling layers used in the BRM in DQN are not sufficiently representative to achieve state-of-the-art quantification results and could benefit from further improvement.

A step further in this direction was the application of differentiable histograms proposed by Pérez-Mon et al. (2024), giving place to a network called HistNetQ. In this work, authors propose a BRM that summarizes the bag computing a histogram for each feature computed by the FEM. This histogram layer represented a generalization of the simple pooling layers used in DQN, providing the BRM with additional information useful for quantification tasks. Since histograms are inherently non-differentiable, the authors propose to use a differentiable approximation using typical deep learning layers as convolutions. This new BRM, along with some innovative techniques for data augmentation in quantification tasks to avoid overfitting when training bags are scarce (see Section 3.5), got very good results in the 2022 quantification competition (Esuli et al., 2022), in particular in the multiclass quantification task (T1B), obtaining the best results so far in the quantification literature (Pérez-Mon et al., 2024).

In the next section, we propose a new BRM based on the use of Gaussian distributions for modeling the latent space computed by the FEM.

3.2. Representing sets using Gaussian distributions in a latent space

In this study, we propose a novel method for obtaining invariant bag features using learnable multivariate Gaussian distributions. The core idea behind this representation layer is to enable the network to model the latent space using a set of multivariate Gaussian distributions where each example's projection in the latent space (bounded in a hypercube

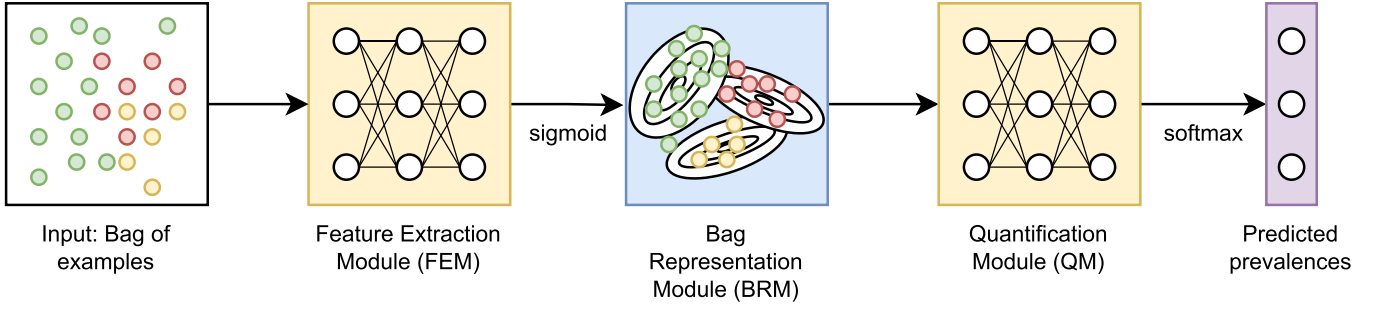


Fig. 1. Basic network architecture for a problem with three classes. The network processes a bag of examples and projects it into a latent space through a Feature Extraction Module (FEM). Then, an order invariant representation of the bag is computed by the Bag Representation Module (BRM). In the figure, Gaussian distributions are used for obtaining the bag representation (as explained in Section 3.2). Other permutation invariant layers as basic pooling layers Qi et al. (2021) or differentiable histograms Pérez-Mon et al. (2024) might be used instead to represent the bags. Finally, the Quantification Module (QM), composed by a set of fully connected layers, relates the bag representation vector to the actual class prevalences. Note that the FEM can be adapted to the problem at hand (CNN for images, Transformers for text, etc.).

of size 1 by a sigmoid activation function) is evaluated against these distributions.

The intuition behind using Gaussians is that they capture complex, multidimensional relationships between features simultaneously, rather than focusing on individual features, simple aggregate statistics (Qi et al., 2021) or differentiable histograms (Pérez-Mon et al., 2024). By modeling the latent space with Gaussians, the network can identify distinct regions within the space where examples are likely to cluster based on feature patterns relevant to quantification. Similar Gaussian-based representations have proven effective in other domains, such as diffusion models for data generation Blessing et al. (2025), video processing Sun et al. (2024) and probabilistic feature embeddings Shi and Jain (2019), suggesting that Gaussians are a suitable choice for capturing complex feature relationships.

In contrast to feature-wise histograms, such as those used in HistNetQ, which summarize each latent feature independently through discretized bin counts, our approach models the joint distribution of features using multivariate Gaussians. Histograms ignore correlations between features and treat each dimension as statistically independent, resulting in a potentially fragmented view of the latent space. In contrast, Gaussian distributions capture covariance across dimensions, enabling the model to represent dependencies and interactions between features. This richer representation allows for more expressive and adaptive modeling of the latent space structure that helps create a more nuanced, continuous representation of the data that is adaptable to shifts in prevalence, ultimately leading to more accurate prevalence estimates.

Let $\mathbf{Z} = [\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_n]$ denote the latent space representations of n examples in a training bag, where each $\mathbf{z}_i \in \mathbb{R}^d$ represents a vector of d features (latent space dimension). Suppose we model the latent space using K Gaussian distributions where $\boldsymbol{\mu}_k$ and $\boldsymbol{\Sigma}_k$ are network learnable parameters representing the mean vector and covariance matrix of the k -th Gaussian distribution.

For each Gaussian k , the likelihood $p(\mathbf{z}_i|k)$ for example i represents how well $\mathbf{z}_i$ fits Gaussian k :

$$p(\mathbf{z}_i|k) = \frac{1}{\sqrt{(2\pi)^d |\boldsymbol{\Sigma}_k|}} \exp\left(-\frac{1}{2}(\mathbf{z}_i - \boldsymbol{\mu}_k)^T \boldsymbol{\Sigma}_k^{-1} (\mathbf{z}_i - \boldsymbol{\mu}_k)\right). \quad (7)$$

To obtain the mean likelihood across all examples in the bag for Gaussian k , denoted by $\bar{p}(k)$, we compute:

$$\bar{p}(k) = \frac{1}{n} \sum_{i=1}^n p(\mathbf{z}_i|k). \quad (8)$$

This mean likelihood $\bar{p}(k)$ provides an aggregate measure of how representative Gaussian k is for the examples of the entire bag. After computing $\bar{p}(k)$ for each Gaussian k , the final representation $\mathbf{r}$ of the bag

is:

$$\mathbf{r} = [\bar{p}(1), \bar{p}(2), \dots, \bar{p}(K)] \in \mathbb{R}^K. \quad (9)$$

In this case, $\mathbf{r}$ encapsulates the averaged likelihoods for each Gaussian, providing a compact invariant representation of the bag in the latent space. The size of this representation will be equal to the number of Gaussian distributions K used to model the latent space.

It is important to note that we can expand this layer by working with a number L of different latent spaces expanded in width (see Fig. 2). The full representation $\mathbf{r}$ of the bag is computed by concatenating the representations from all latent spaces:

$$\mathbf{r} = [\mathbf{r}_1, \mathbf{r}_2, \dots, \mathbf{r}_L] \in \mathbb{R}^{LK}. \quad (10)$$

Here, $\mathbf{r}$ combines all individual representations into a single vector, providing a comprehensive representation of the bag across multiple latent spaces. In this particular case, the FEM is replicated across every latent space, to make the projection unique and different for each of them (see Fig. 2).

This bag representation $\mathbf{r}$ is then used by the QM (see Fig. 1) to predict a final prevalence vector for the bag in question. As training progresses the network will back-propagate the loss function (a quantification error measure like the ones defined in Section 2.2 that will work as a loss function) and learn to place and shape the Gaussian distributions so the BRM is capable of providing a good representation for the QM to predict good prevalence estimates (see Fig. 3).

The difference between this BRM and those based on basic pooling functions like *average*, *max*, or *median* is substantial, as it provides a more expressive representation than merely computing a single statistic per feature. While the network can adapt its layers to leverage the available information, providing a better embedding for the bag aids significantly in converging to an optimal solution. The distinction from a histogram layer is more subtle. Firstly, histograms are inherently discrete, whereas Gaussian distributions offer a continuous representation that avoids binning. Secondly, creating a histogram per feature in the latent space loses the connections between features, whereas Gaussian distributions may capture these correlations by considering all features simultaneously, resulting in a richer and more accurate representation.

Mathematically, this advantage stems from the off-diagonal elements of the learnable covariance matrix $\boldsymbol{\Sigma}_k$ defined in Eq 7. While a histogram-based approach is restricted to modeling the univariate marginals of each of the d dimensions, our Gaussian approach models the full joint density of the latent space. By learning the covariance between dimensions, GMNet can find patterns in example clusters, patterns that would be invisible to feature-wise histograms which assume feature independence. Furthermore, the likelihood $p(\mathbf{z}_i|k)$ provides a continuous,

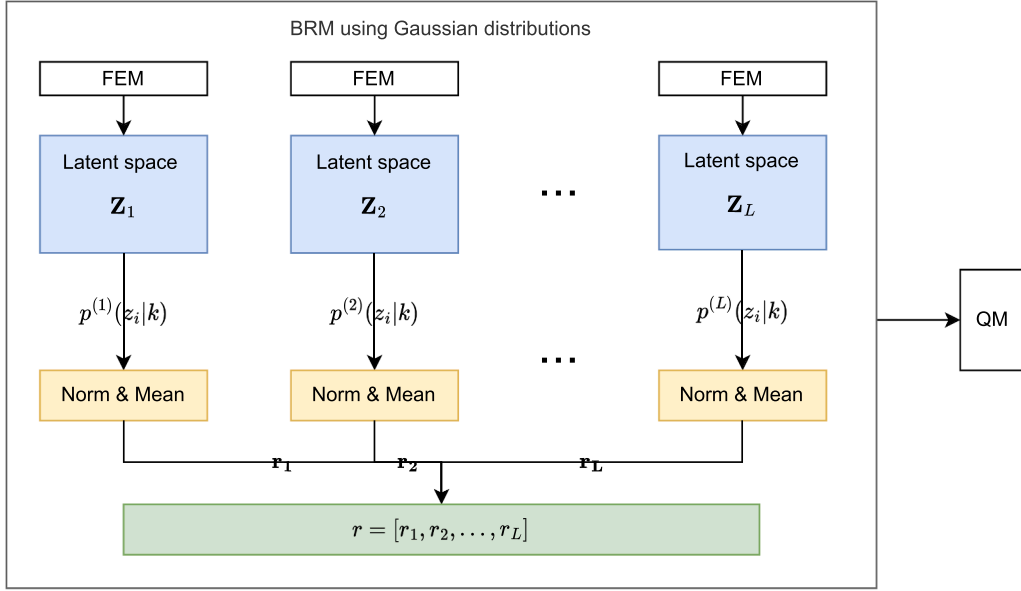


Fig. 2. Extension for using multiple latent spaces representations. Each latent space Z_i can have a different number of dimensions d , and a different number K of Gaussian distributions to model it (hyperparameters). After the examples in the bag are projected to each latent space, we compute the likelihood $p^{(i)}(z_i|k)$ of each example i to belong to each Gaussian distribution $k = 1 \dots K$ in a latent space $l = 1 \dots L$. Then we apply normalization and the mean by Gaussian, getting a vector r_l . The final representation layer is the concatenation of each vector r_l .

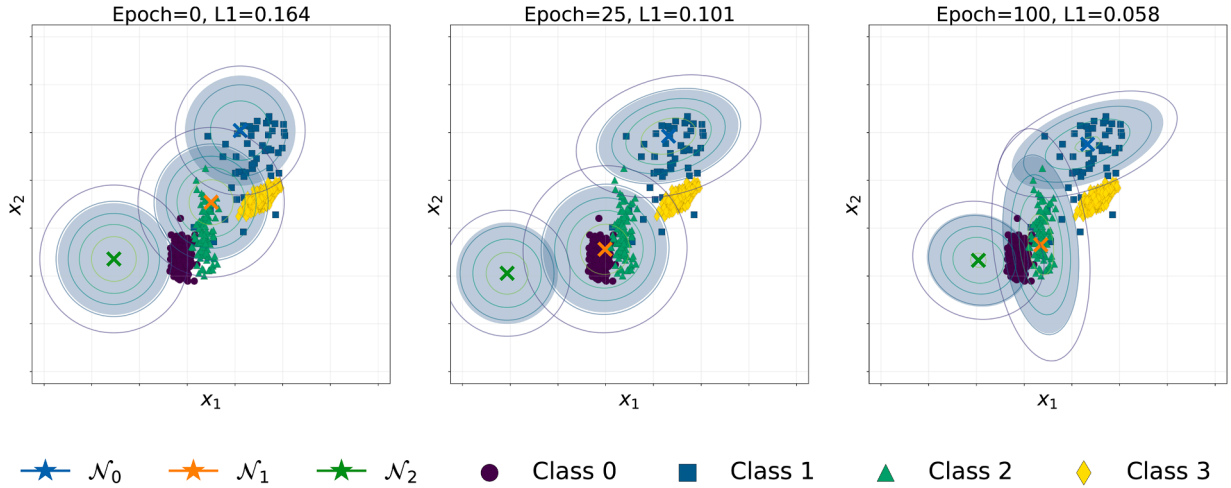


Fig. 3. Example of the BRM inner workings using a bare-bones architecture. Input data corresponds to a synthetic multiclass problem with four classes and two input dimensions. Data is projected to a latent space just using a *sigmoid* function (note that in the full network, the FEM will be in charge of projecting input data to the latent space). While the experimental setup uses 100 Gaussians, only 3 are shown here for visualization purposes to maintain clarity. As training progresses, the network learns to place the Gaussian distributions in convenient places to create a useful representation r of the bag, that the QM can use with the objective of minimizing the chosen loss function (L1 in this case). Note that the number of classes and the number of Gaussian distributions are not directly related, as the network will learn their centers and shapes to optimally represent the data.

smooth signal for backpropagation, avoiding quantization errors inherent to discrete binning.

3.3. Parameter initialization

Parameter initialization is a key ingredient for successfully training a neural network. In the architecture presented, the linear and batch normalization layers of the FEM and QM follow the default initialization procedures used in the PyTorch framework, which are standard for ensuring stable gradient flow. However, each representation layer in the BRM requires a specific approach to place a set of K Gaussian

distributions in the latent space. For each Gaussian distribution, its center μ_i and its covariance matrix Σ_i are network parameters that require initialization.

The BRM is preceded by a *sigmoid* function, which confines the latent space to $[0, 1]$, resembling a hypercube of dimension d . Our design choice was to initialize the Gaussian centers μ_i uniformly distributed across all dimensions within the range $[0, 1]$.

Regarding the initial covariance matrix Σ_i , we adopt a diagonal matrix of size d (the dimensionality of the latent space). The diagonal elements Σ_{ii} are initialized based on a heuristic involving pairwise distances between Gaussian centers μ_i . Specifically, we compute these

distances (excluding self-distances) and use the average minimum distance divided by 2, squared:

$$\Sigma_{ii} = \left(\frac{\text{mean}(\min_{i \neq j} |\mu_i - \mu_j|)}{2} \right)^2. \quad (11)$$

The intuition behind this heuristic is to make the Gaussians cover the maximum space possible in the hypercube without excessive overlapping.

As each element of the covariance matrix Σ_i serves as a parameter in the network, it is crucial to ensure that Σ_i remains a valid covariance matrix, meaning it must be positive-definite during all the training process. To achieve this, we utilize *geotorch* (Lezcano-Casado, 2019), a library that enables us to enforce the positive-definiteness of each Σ_i . This constraint is essential for maintaining the integrity of Gaussian distributions in our model, supporting robust training and reliable inference.

The rest of the network parameters (linear layers, batch norm layers, etc.) are initialized with the default procedure used in the PyTorch deep learning framework.

3.4. Latent space similarity regularization

As explained in Section 3.2, our network architecture is extended horizontally to incorporate multiple latent space representations, aiming to capture diverse information about each bag. Each latent representation operates with its own set of Gaussian distributions to model and derive a robust invariant representation essential for quantification tasks.

The network by itself, when trying to optimize for the loss provided, will try to make use of these different latent space representations to make a powerful joint representation. A way of helping the network in this process could be enforcing that the latent spaces differ among them. To compute the similarity between latent spaces we have used the score provided by Centered Kernel Alignment (CKA) (Kornblith et al., 2019), which is able to compute the similarity of latent spaces of any dimension.

The CKA score measures the similarity between representations learned by different neural networks even with different dimensions provided that they are evaluated with the same set of examples. Formally, given a set of L latent space activations $\{\mathbf{Z}_1 \in \mathbb{R}^{n \times d_1}, \mathbf{Z}_2 \in \mathbb{R}^{n \times d_2}, \dots, \mathbf{Z}_L \in \mathbb{R}^{n \times d_L}\}$, where n is the number of examples and d the dimension of each latent space, the CKA score is computed as follows:

$$\text{CKA}(\{\mathbf{Z}_i\}) = \frac{1}{\binom{L}{2}} \sum_{i < j} \frac{\|\mathbf{Z}_i^\top \mathbf{Z}_j\|_F^2}{\|\mathbf{Z}_i^\top \mathbf{Z}_i\|_F \|\mathbf{Z}_j^\top \mathbf{Z}_j\|_F}, \quad (12)$$

where $\|\cdot\|_F$ denotes the Frobenius norm. The numerator computes the similarity between representations $\mathbf{Z}_i$ and $\mathbf{Z}_j$ while the denominator serves to normalize the similarity to ensure it is not influenced by the scale of the activations, thereby providing a scale-invariant measure of alignment between the latent spaces.

By incorporating this similarity metric into the loss function with an appropriate weighting factor (a hyperparameter), the network is compelled to converge towards solutions where the latent spaces exhibit greater differentiation and consequently offer more informative representations. Our experimental findings show that integrating this approach sometimes improves not only the results but also accelerates network convergence.

The CKA regularization term is added to the total loss of the network before backpropagation, multiplied by a regularization factor λ . The modified loss function is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{original}} + \lambda \cdot \text{CKA}(\{\mathbf{Z}_i\}), \quad (13)$$

where $\mathcal{L}_{\text{original}}$ is the original loss function of the network (usually a quantification loss, such as those defined in Section 2.2), λ is the regularization factor, and $\text{CKA}(\{\mathbf{Z}_i\})$ is the Centered Kernel Alignment score computed over the set of latent space projections $\{\mathbf{Z}_i\}$.

3.5. Data augmentation

Every deep learning system is highly dependent on the amount of data available. For quantification, the ideal scenario involves having bags of examples labeled by prevalence. In a symmetric setting, even with a large number of labeled bags, each bag in quantification becomes a single training example, which makes typical dataset sizes relatively small for deep learning. For instance, in our experiments, one thousand labeled bags by prevalence represent a low number of training examples for a neural network, making data augmentation techniques essential.

In the context of quantification, data augmentation can be achieved by mixing bags to create new ones (Pérez-Mon et al., 2024). This approach is particularly useful when the number of labeled bags is limited. By randomly picking and mixing two real bags, we can produce a new *augmented* bag. The prevalence of the new bag is computed as the mean class prevalences of the two mixed bags. This data augmentation technique is called "Bag Mixer" and will be used in the experiments for all deep learning methods.

It is important to note that when individual example labels are available alongside bags labeled by prevalence, they can be leveraged to generate additional training bags. In this scenario, we can generate new synthetic training bags with dataset shift for training deep quantifiers. For instance, to create new bags with prior probability shift, we can just make a grid and change the class priors of each class step by step. While this procedure might work well for two classes, it gets unfeasible when the number of classes increases. The alternative is to use an algorithm like Kraemer (Smith & Tromble, 2004) to draw uniform prevalence vectors at random and then obtain the bags matching these prevalence values using sampling with replacement from the training dataset. This protocol for generating bags is usually known in the quantification literature as APP protocol (Esuli et al., 2023). As we will demonstrate in the experiments and in consonance with previous research in deep learning, the more amount and variety of data provided to the network, the better its performance and convergence (a result that, on the other hand, was expected).

4. Experiments

4.1. LeQua datasets

The first group of datasets used in these experiments are part of the only two quantification competitions held up to date (LeQua, editions 2022 (Esuli et al., 2022) and 2024¹). The chosen datasets for this study correspond to multiclass (with names T1B for 2022 and T2 for 2024) and ordinal quantification (with name T3), as they are the more challenging and the ones that pose bigger difficulties to quantification methods. All datasets are freely available for download and consist of product reviews extracted from Amazon.

For multiclass quantification tasks (T1B and T2), each review belongs to one of 28 different merchandise classes ("Automotive", "Baby", "Beauty", ...). Our task is to predict the class prevalences of bags of reviews affected by prior probability shift. The only difference between T1B and T2 is the input space of the features. T1B provides 300 numeric features per review while T2 provides only 256. The objective function for these tasks is Relative Absolute Error (RAE) (see Section 2.2).

The ordinal quantification task consists of predicting the prevalence of each class, in this case, sentiment-based polarity, expressed as a star rating (from "1 star" to "5 stars") in bags of product reviews. The particularity of this task is that classes have implicit order, which means that mistakes do not count the same when predicting the class prevalences. The loss function used for this task is Normalized Matched Distance (NMD) (see Section 2.2).

¹ <https://lequa2024.github.io>

All three datasets provide 20,000 individual labeled examples, along with 1000 bags labeled by prevalence. Each training bag contains 1000 individuals for T1B and T2, and 200 individuals for T3. It is important to note that these bags are labeled by prevalence but the label of each example in them is not available. The test set comprises 5000 bags for each task, each containing 1000 elements for T1B and T2, and 200 for T3.

4.1.1. Experimental study

This section describes the experiments in the three LeQua datasets. Two main approaches were employed: traditional quantifiers, trained with the QuaPy public quantification library (Moreo et al., 2021), and quantification methods based on deep learning. On the one hand, traditional quantifiers, as detailed in Section 2, require a classifier that was trained using datasets with individual example labels. This means that training bags labeled by prevalence cannot be used in the training process directly. Instead, they are used to perform a grid search to optimize classifier and quantifier hyperparameters, aiming for the best quantification performance based on official evaluation metrics (RAE for T1B and T2, and NMD for T3). On the other hand, quantification methods based on deep learning are optimized based on a chosen loss function (as before, RAE, for T1B and T2 and NMD for T3).

For the three datasets, two different setups were considered for the deep learning lot:

- **U (unlabeled) setting.** In this scenario, deep learning methods use only provided bags for training, discarding the labeled dataset. 70% of the available bags labeled by prevalence (700 bags) were used for training, while the other 30% (300 bags) were used for validation and early stopping.
- **U + APP setting.** In this scenario, apart from 700 training bags, we feed the network with extra synthetically generated training bags presenting prior probability shift, using the APP protocol (see Section 3.5) over the individually labeled examples. In this setup, APP-generated bags represent the 50% of all the bags fed to the network.

In both settings, bag-labeled data was augmented by randomly mixing real bags using the *Bag Mixer* described in Section 3.5. Running the experiments without the *Bag Mixer* was discarded because the ablation study in Pérez-Mon et al. (2024) demonstrates that deep learning quantification methods suffer significantly from overfitting without this data augmentation strategy.

While the hyperparameters of traditional quantification were optimized using the validation dataset, an exhaustive hyperparameter search for deep learning methods was not feasible due to limited resources. The size of the networks was adjusted to fit within a single small GPU (with 24 GB of RAM). All deep learning models were trained using the same architecture, with the only difference being the BRM module used. This setup was explicitly designed to isolate and evaluate the impact of each BRM on quantification performance. In the BRM module we tested basic pooling layers -such as average, median, and max- referred to as DQN (Qi et al., 2021); differentiable histograms, named HistNetQ (Pérez-Mon et al., 2024); and the layer proposed in this paper based on multivariate Gaussian distributions, called GMNet (see Section 3.2). The number of Gaussian distributions was set to 100 for each latent space, with each space in 5 dimensions. We used 9 latent spaces. The value for the CKA regularization was fixed to 0.01 for all experiments. For HistNetQ, we used a histogram with 32 bins per feature.

The FEM utilized in these experiments comprises a series of fully-connected layers with dropout, given that all three datasets used consist of tabular data. The last layer of the FEM, which corresponds to the input of the BRM had a size of 512. It should be noted that in GMNet, with latent spaces, the FEM is integrated within each submodule to obtain different projections (see Fig. 2). In this case, we chose an output size for each FEM in each submodule of 50 neurons. In terms of the QM, all networks use the same set of fully connected layers with dropout

followed by a softmax activation function, which will finally output the estimated prevalence of the classes.

All networks were trained using an identical procedure to ensure comparability². We applied early stopping after 40 epochs in which the validation loss did not improve.

In Table 1, we present the performance of the quantification methods on the three datasets. A detailed analysis of the results for the T1B dataset reveals that GMNet outperforms all other methods, including traditional and deep learning-based ones, in both the U and U + APP settings. This is particularly notable as GMNet significantly surpasses HistNetQ, the previously best-known method for this quantification task (Pérez-Mon et al., 2024). The difference between U + APP and U setups is notable as all deep learning methods improve their results when also using bags generated with APP. This underscores how deep learning methods benefit from additional training data. Among the deep learning methods, the ones with more complex BRMs (histograms and Gaussian distributions) stand out as the best compared to methods with simpler representations as median or average, that fall back but perform exceedingly well compared with traditional methods. Among traditional methods, EMQ without calibration stands out as the best performer. These results highlight the advantage of deep learning methods in optimizing specific loss functions and the ability to train directly using bags, compared to traditional quantification methods, which do not have these capabilities.

Concerning the T2 dataset, the same tendencies are observed as for T1B. GMNet comes out as the best performer and the best deep learning methods (GMNet, HistNetQ, and DQN (med)) outperform by a wide margin all traditional methods, for the optimized loss (RAE), including EMQ, which is considered one of the best quantification methods in the literature (Alexandari et al., 2020). When the amount of data is reduced (U setting) the performance of the deep learning methods systematically decreases.

In the T3 dataset, deep learning methods once again demonstrate superior performance, but now with HistNetQ slightly ahead of GMNet. The capability to effectively optimize loss functions designed for ordinal quantification further strengthens the advantage of these methods in tackling such tasks. It is interesting to note that in this dataset, even the BRM that consistently gets the worst results among the deep learning lot (*max* pooling layer), outperforms all traditional methods in the U + APP setting.

4.1.2. Impact of data availability on quantification methods

The previous section demonstrates that, in the context of the LeQua quantification competition, there exists a substantial performance gap between the best deep learning methods (HistNetQ and GMNet) and traditional quantification methods in the three tasks analyzed. However, to better understand the underlying reasons for this disparity, we designed an additional experiment aimed at answering, in our opinion, two key questions: i) Is the gap primarily due to the inability of traditional methods to exploit the same training data as deep learning methods? ii) How does the amount of training data affect the performance of deep learning methods in quantification tasks?

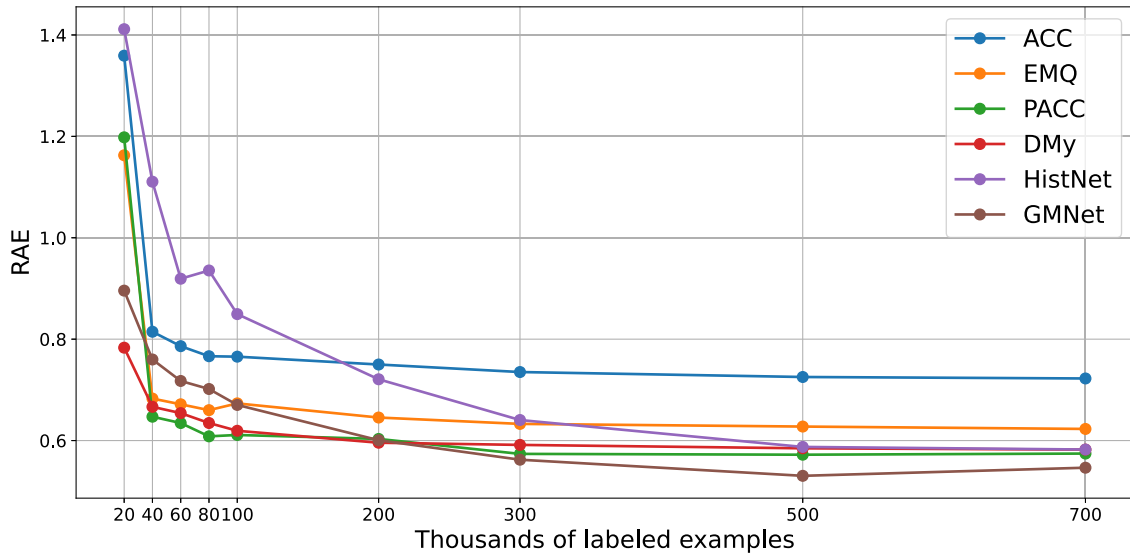
In the main experiment of the paper (see Section 4.1.1), deep learning methods are able to exploit bags labeled by prevalence for training, a capability that traditional methods lack (although, they use this data for validation and hyperparameter tuning). To perform this experiment, we requested the individual labels of the 1000 bags labeled by prevalence of dataset T2 to the organizers of the quantification competition LeQua2024, enabling traditional quantifiers to train using this data. This new dataset comprises 1 million labeled examples. We conducted experiments with different training-validation splits incrementally, from 20k to 700k training examples. The same two settings as in the main exper-

² Code and data to fully reproduce the experiments can be found in <https://github.com/AICGijon/gmnet>

Table 1

Results for T1B, T2 and T3, in terms of RAE (for T1B and T2) and NMD (for T3). Deep learning methods were trained using only bags labeled by prevalence (U) and bags labeled by prevalence plus synthetic bags generated using the APP protocol (U + APP).

	T1B (RAE)	T2 (RAE)	T3 (NMD)
CC	1.8936 $\pm$ 1.187	2.3096 $\pm$ 1.384	0.0809 $\pm$ 0.047
PCC	2.2646 $\pm$ 1.416	2.6488 $\pm$ 1.610	0.0669 $\pm$ 0.051
ACC	1.4213 $\pm$ 1.270	1.3480 $\pm$ 1.164	0.1193 $\pm$ 0.065
PACC	1.3054 $\pm$ 0.988	1.1956 $\pm$ 1.137	0.1246 $\pm$ 0.066
DMy	1.0419 $\pm$ 1.030	1.3189 $\pm$ 1.128	0.1156 $\pm$ 0.065
EMQ-NoCalib	0.8780 $\pm$ 0.751	1.1623 $\pm$ 0.991	0.1130 $\pm$ 0.070
EMQ-Platt	1.1959 $\pm$ 1.137	1.9203 $\pm$ 1.900	0.1806 $\pm$ 0.099
DQN (avg) - U	0.9586 $\pm$ 0.629	1.3169 $\pm$ 0.936	0.0552 $\pm$ 0.041
DQN (max) - U	1.3163 $\pm$ 0.882	2.0445 $\pm$ 1.357	0.2247 $\pm$ 0.060
DQN (med) - U	0.7893 $\pm$ 0.499	1.1779 $\pm$ 0.847	0.0546 $\pm$ 0.040
HistNetQ - U	0.6989 $\pm$ 0.451	0.9162 $\pm$ 0.598	0.0489 $\pm$ 0.036
GMNet (ours) - U	0.6728 $\pm$ 0.368	0.8816 $\pm$ 0.550	0.0502 $\pm$ 0.037
DQN (avg) - U + APP	0.7970 $\pm$ 0.524	1.3105 $\pm$ 0.968	0.0537 $\pm$ 0.040
DQN (max) - U + APP	0.9211 $\pm$ 0.571	2.4288 $\pm$ 1.729	0.0587 $\pm$ 0.043
DQN (med) - U + APP	0.7286 $\pm$ 0.466	1.0852 $\pm$ 0.815	0.0534 $\pm$ 0.039
HistNetQ - U + APP	0.5711 $\pm$ 0.374	0.7455 $\pm$ 0.501	0.0467 $\pm$ 0.036
GMNet (ours) - U + APP	0.5433 $\pm$ 0.263	0.7062 $\pm$ 0.420	0.0498 $\pm$ 0.037

**Fig. 4.** U + APP setting over the T2 dataset.

iment were considered for the deep learning methods (U and U + APP). In the U setting, only bags labeled by prevalence are used for training deep learning methods while traditional methods will use all the examples in these bags along with their labels. In the U + APP setting, the label of the examples in the training bags is used also for generating new synthetic training bags with prior probability shift (using the APP protocol) that are fed to the network along with the training bags.

Fig. 4 presents the results for the U + APP setting, while Fig. 5 shows the results for the U setting on the T2 dataset using different training-validation splits. For clarity, only the best-performing methods of each type are included in these experiments.

As expected, results show that traditional methods benefit significantly from having more than 20k labeled examples. Training a classifier to estimate probabilities in a 28-class problem is highly challenging with only 20k examples (i.e. ~ 710 examples per class). Results show that with 100k examples traditional methods obtain already very good results, with only marginal improvements beyond this point.

On the other hand, deep learning methods, such as HistNetQ and GMNet, tend to perform better as the amount of data increases, a trend

expected when using deep learning. The results indicate that GMNet, using the U + APP setting, becomes competitive with the best traditional quantification methods when trained on 200k examples (200 training bags), with its performance continuing to improve as the dataset size increases. Conversely, HistNetQ needs more training data to be competitive, not matching the best traditional methods until 500 training bags are provided. In the U setting (see Fig. 5), deep learning methods only use a very limited number of training bags (augmented with the *Bag Mixer*). Even with 700 training bags, the results obtained are not comparable to those achieved by traditional methods using 700k labeled examples. For instance, when using 20k examples, we are only providing the network with 20 different bags (each bag has 1000 examples), while attempting to estimate prevalences in a 28-class problem, making the task almost impossible. Note that APP (used in the U + APP setting) greatly helps the network introducing a lot of variability in the training bags, that are synthetically created with different examples (picked at random) and different prevalences (generated using the Kraemer algorithm), covering the entire prevalence space. APP is the only mechanism by which these systems leverage individual example labels. In the U

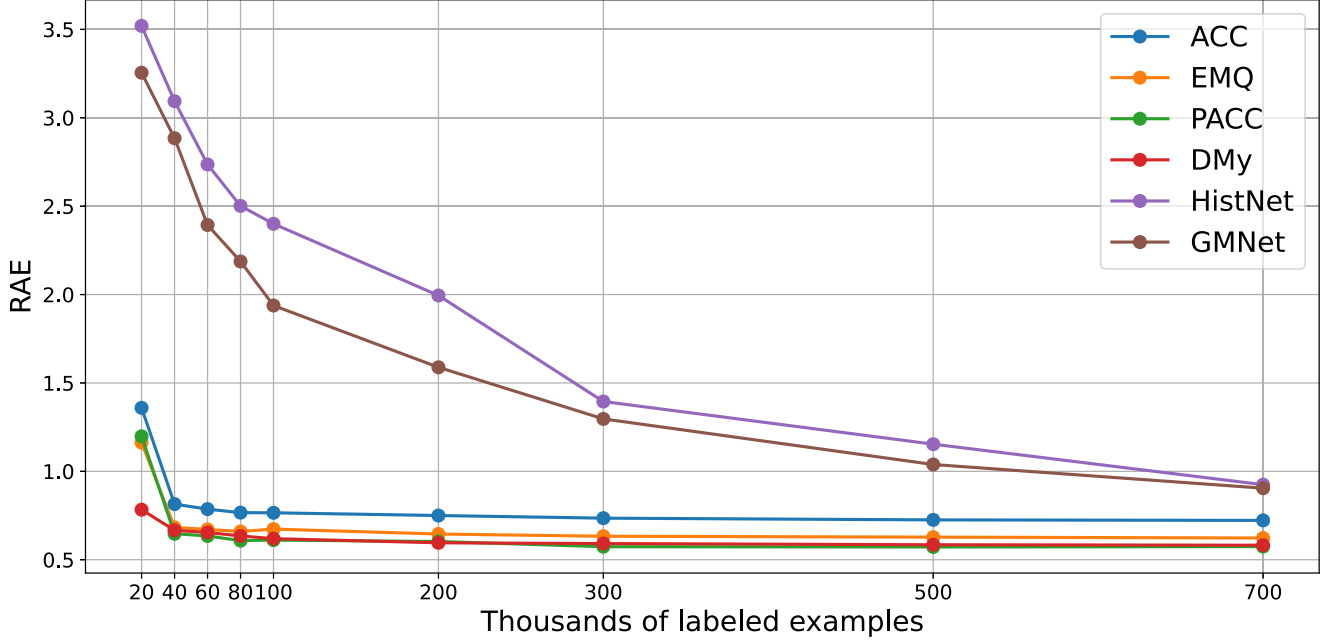


Fig. 5. U setting over the T2 dataset.

Table 2

Results for GMNet using different λ values controlling the CKA regularization.

λ	T1B (RAE)	T2 (RAE)	T3 (NMD)
0	0.5531 $\pm$ 0.272	0.7130 $\pm$ 0.392	0.0485 $\pm$ 0.036
0.1	0.5307 $\pm$ 0.286	0.7113 $\pm$ 0.361	0.0507 $\pm$ 0.038
0.01	0.5433 $\pm$ 0.263	0.7062 $\pm$ 0.420	0.0498 $\pm$ 0.037
0.001	0.5345 $\pm$ 0.288	0.7113 $\pm$ 0.388	0.0492 $\pm$ 0.035

setting, only the available training bags are fed to the network. This setup is disadvantageous for the networks as the amount of training instances is very limited.

In Table 1, HistNetQ results may appear quite similar to those of GMNet. This experiment highlights that GMNet exhibits a reduced dependence on data volume compared to HistNetQ, remaining competitive even with limited data. The performance gap between the two models is substantial when using small amounts of training data but narrows as more data becomes available to the networks.

4.1.3. Latent space regularization

To evaluate how latent space regularization (see Section 3.4) affects GMNet, we conducted various experiments applying different values of CKA regularization (λ) in all three datasets, keeping the rest of the training hyperparameters untouched. The results are presented in Table 2. On the T1B and T2 datasets, CKA regularization enhances performance to some degree, while on T3, the tested values of λ yield no observable improvement. In this case, the BRM may play a less critical role, as even a simple pooling mechanism like *max* achieves competitive results.

In summary, these results show that regularization may help improve performance, helping the network to converge to solutions with different latent spaces that provide richer information. As with any other hyperparameter, the correct value must be set for each problem. Even though the margin is not wide, it usually provides faster convergence time that pays off the extra amount of computation needed for calculating the CKA term (less than 1% of the computing time needed for completing an epoch).

4.2. Experiments on CIFAR-10 and CIFAR-100coarse datasets

To demonstrate the performance of GMNet beyond the tabular LeQua competition datasets, we conducted additional experiments using the CIFAR-10 and CIFAR-100 datasets, which are widely used benchmarks in the machine learning community.

CIFAR-10 consists of 60,000 color images (32×32 pixels) divided into 10 distinct balanced classes. CIFAR-100 contains the same number of images (with the same size) but is organized into 100 fine-grained classes. For our experiments, we utilized the 20 coarse labels (super-classes) of CIFAR-100. This design choice corresponds to the fact that generating representative bags for 100 classes is problematic; it would require significantly larger bag sizes to avoid extreme sparsity, and the dataset lacks sufficient examples per class to support such a configuration reliably.

For both datasets, we used 50,000 images for training (reserving 10,000 for validation) and the 10,000-image test set to generate 1000 evaluation bags. Each bag contained 500 examples and was generated following the APP protocol to present prior probability shift.

Given that these datasets consist of image data rather than tabular features, we first trained a simple Convolutional Neural Network (CNN) on the respective training sets to act as a feature extractor. This ensures that all quantification models (both traditional and deep learning-based) are trained on identical input features. This focuses the evaluation strictly on the quantification performance of each method rather than the feature extraction process. The CNN architecture utilized for this purpose outputs a 256-dimensional feature vector for each image.

As in Section 4.1.1, two main approaches were employed: traditional quantifiers, trained using the QuaPy public quantification library (Moreo et al., 2021), and deep learning-based quantification methods. For the deep learning methods, we followed the same experimental process as in the LeQua datasets. Specifically, all networks were trained using identical architectures and hyperparameters (the same used in the LeQua experiments), varying only the BRM component. Since no unlabeled bags were available in this setting, we applied the APP protocol to generate synthetic training bags.

In Table 3, we observe that the best-performing method is GMNet, followed closely by EMQ, DMy and the other deep learning methods.

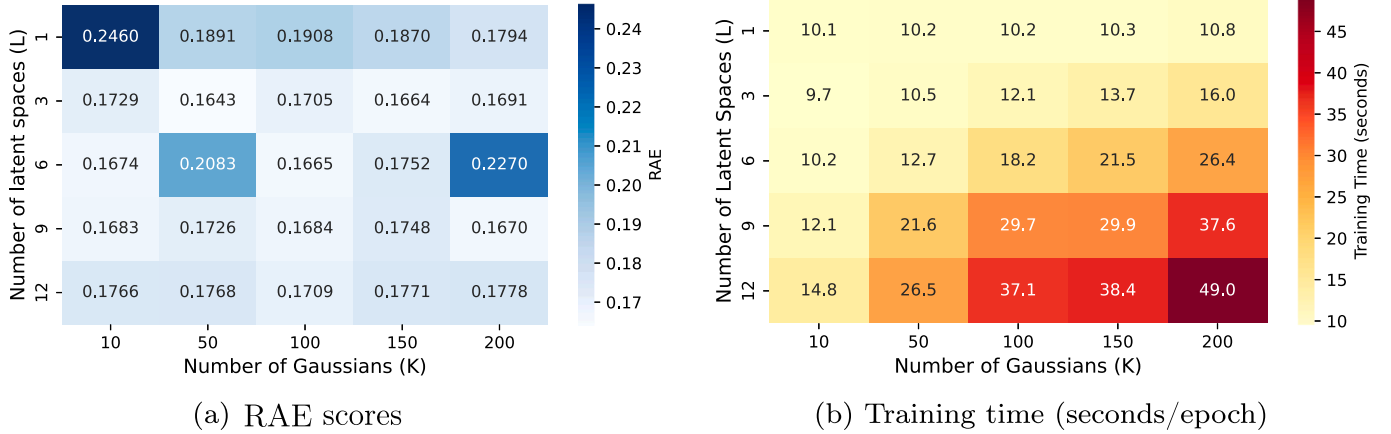


Fig. 6. Impact of K and L hyperparameters on GMNet performance (RAE) and computational cost (seconds per epoch) using the CIFAR-10 dataset (total training time is not shown as it depends on the total number of epochs needed for convergence but it can be considered proportional to these values).

Table 3

Performance (RAE), training time, and inference time of all methods on the CIFAR-10 and CIFAR-100 datasets. GMNet achieves the best performance. Training and inference times illustrate the computational cost associated with each approach.

Method	CIFAR-10	TT	IT	CIFAR-100	TT	IT
CC	0.5296 ± 0.696	25s	17s	0.8977 ± 0.5800	35s	18s
ACC	0.3602 ± 0.476	175s	53s	0.8022 ± 0.5277	192s	67s
PCC	0.6084 ± 0.745	20s	17s	0.9582 ± 0.6029	31s	17s
PACC	0.3558 ± 0.418	174s	41s	0.7959 ± 0.5057	190s	67s
DMY	0.2144 ± 0.234	171s	38s	0.4893 ± 0.3143	197s	77s
EMQ	0.1886 ± 0.205	20s	18s	0.4889 ± 0.3151	31s	21s
EMQ-Platt	0.5029 ± 0.597	144s	23s	0.5787 ± 0.2480	154s	25s
DQN (avg)	0.2211 ± 0.222	1h20m	20s	0.5620 ± 0.3666	1h39m	20s
DQN (max)	0.9391 ± 0.643	59m	20s	0.7270 ± 0.3767	1h38m	20s
DQN (med)	0.2207 ± 0.239	1h15m	20s	0.6656 ± 0.4438	1h57m	20s
HistNetQ	0.2271 ± 0.248	2h2m	21s	0.6884 ± 0.4194	2h4min	21s
GMNet (ours)	0.1684 ± 0.123	2h57m	21s	0.3884 ± 0.2331	4h49m	21s

These results are somewhat expected given the limited amount of training data available, which makes these tasks more challenging for deep learning methods that inherently benefit from large datasets (see Section 4.1.2). Notably, GMNet exhibits the lowest variance among all methods, indicating particularly stable and robust predictions.

4.2.1. Computational cost analysis

To address the computational cost of deep learning methods, especially GMNet, we include a comparison of training and inference times for all evaluated methods (see Table 3). As expected, deep learning-based quantifiers require significantly longer training times than traditional quantifiers, due to their symmetric training approach, which involves feeding the network with training bags labeled by prevalence. However, deep learning methods achieve competitive inference times, as they can process bags in batches, unlike traditional methods that handle bags sequentially.

When comparing GMNet with the other deep learning approaches, GMNet requires more training time due to its more complex representation, which incorporates multiple latent spaces and Gaussian modeling. Nevertheless, the difference in training time is relatively small and not only attributable to architectural complexity, since the number of training epochs is not fixed but determined by an early stopping process. Despite the higher training cost, GMNet consistently achieves superior quantification performance. This suggests that the increased expressiveness of the GMNet architecture offers meaningful advantages with only a moderate increase in training time and no substantial difference in inference time.

4.2.2. Impact of K and L hyperparameters

To address the sensitivity of the proposed architecture to its primary hyperparameters, we conducted an empirical exploration on the CIFAR-10 dataset by varying the number of Gaussian distributions (K) and the number of latent spaces (L). As noted in Section 4, the values of $K = 100$ and $L = 9$ were originally selected to ensure the network fits within a single GPU with 24 GB of RAM and were kept constant across all the experiments (included LeQua and CIFAR-10) to ensure comparability. The results of this exploration, measured in terms of Relative Absolute Error (RAE), are illustrated in the hyperparameter exploration heatmap (see Fig. 6a).

The results reveal an important interplay between the complexity of individual latent spaces and the total number of spaces used. When the number of Gaussians is low ($K = 10$), the model performs poorly with a single latent space. However, the performance improves significantly as the number of latent spaces increases. This indicates that the network is capable of compensating for a simpler modeling of individual latent spaces by aggregating information across a higher diversity of projections.

Conversely, when the number of latent spaces is small, such as $L = 3$, increasing the number of Gaussians per space from $K = 10$ to $K = 50$ provides a substantial performance boost. The exploration shows that the model remains relatively stable across a wide range of configurations. We observe a localized increase in error at $L = 6$ and $K = 200$, which may suggest a risk of overfitting when the number of learnable parameters becomes excessively high relative to the specific features of the latent space. Overall, these findings demonstrate that GMNet is robust to different configurations of K and L , reaching competitive performance through various combinations of representation width and depth. Complementing the performance analysis, the second heatmap (see Fig. 6b) illustrates the training time in seconds per epoch for different configurations. As expected, there is a clear correlation between the architectural complexity and the computational cost. While configurations with low K and L (e.g., $K = 10, L = 1$) require only approximately 10.1 seconds per epoch, the most complex configuration tested ($K = 200, L = 12$) increases this cost to 49.5 seconds per epoch. Notably, this exploration shows that for the CIFAR-10 dataset a configuration with less latent spaces and less Gaussian components would have been sufficient for obtaining state-of-the-art results with less computational overhead.

5. Conclusions

In this paper, we have presented a neural network for quantification that utilizes Gaussian distributions in latent spaces to obtain invariant

representations of bags of examples, called GMNet. This representation proved useful to capture relationships among features in complex multiclass quantification problems, providing state-of-the-art results in the T1B and T2 datasets, the standard benchmarks for the quantification community in the multiclass setting. Additionally, the inherent ability of deep learning methods to train directly on bags and optimize task-specific loss functions offers significant advantages over traditional quantification methods, which lack these capabilities. This feature is particularly beneficial in problems with highly specific loss functions, such as T3, an ordinal quantification task where the NMD loss function is employed.

A key contribution of GMNet is the design of its bag representation module (BRM), which models the joint distribution of features in latent space using multivariate Gaussians. This contrasts with the previous state-of-the-art deep quantification method, HistNetQ, which uses feature-wise histograms. By capturing feature dependencies in a continuous representation (instead of discretizing each feature independently) GMNet provides a more informative summary of the input bags.

The outstanding experimental results obtained by GMNet highlight some of the advantages of the proposed architecture, including: i) the quality of the proposed bag representation module (BRM), which is able to capture complex interactions between features projected in latent spaces, that improves the results obtained by previous methods; ii) the capability to directly optimize a specific loss function, as demonstrated in T1B and T2 by optimizing RAE, and in T3 by optimizing the ordinal loss NMD; iii) the ability to perform without example-labeled data, which is an interesting capability that traditional quantification methods do not have; iv) its performance with limited data compared to other deep learning quantification methods (HistNetQ); v) the effectiveness of regularization in complex problems, allowing GMNet to solve them more efficiently.

Future work may include extending this study to other datasets, which have proven particularly challenging for traditional quantification methods (González et al., 2019) and where training bags are naturally present. Also, the application of these methods to problems where other types of shift different from prior probability shift occur (González et al., 2024). Additionally, our BRM has potential applications beyond quantification, such as in general set processing tasks (Zaheer et al., 2017), where permutation invariant representations are essential, or Learning from Label Proportions (LLP) (Yu et al., 2014). It also remains to be explored how GMNet performs when modifying the FEM to use task-adapted feature extractors (such as convolutional neural networks for image data as in CIFAR-10 and CIFAR-100). This could allow the network to jointly optimize both feature extraction and quantification, potentially improving performance on more complex domains. Additionally, it could be worth exploring the integration of topological constraints or order-aware components within the Bag Representation Module (BRM), potentially improving results for ordinal quantification tasks like T3 where class proximity is a key factor in the loss function. A final avenue for future work, and maybe the most critical to improving the proposed method, could focus on how to make better use of the data available, as for instance new data augmentation techniques or other forms of synthetic training bag generation.

CRediT authorship contribution statement

Olaya Pérez-Mon: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Juan José del Coz:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Pablo González:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Project administration, Methodology, Investigation, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work has been funded by Agencia de Ciencia, Competitividad Empresarial e Innovación Asturiana (Sekuens) within the program “Grants for research groups from organizations in the Principado de Asturias, 2024” (GRU-GIC-24-018).

The authors would like to thank Dr. Alejandro Moreo and Dr. Pablo Pérez for their insightful discussions about the paper.

References

- Alexandari, A., Kundaje, A., & Shrikumar, A. (2020). Maximum likelihood with bias-corrected calibration is hard-to-beat at label shift adaptation. In *International conference on machine learning* (pp. 222–232). PMLR.
- Bella, A., Ferri, C., Hernández-Orallo, J., & Ramírez-Quintana, M. J. (2010). Quantification via probability estimators. In *2010 IEEE International conference on data mining* (pp. 737–742). ISSN: 2374-8486. <https://doi.org/10.1109/ICDM.2010.75>
- Blessing, D., Jia, X., & Neumann, G. (2025). End-to-end learning of gaussian mixture priors for diffusion sampler. In *The thirtieth international conference on learning representations*. <https://openreview.net/forum?id=ixBUquaWbl>.
- Castano, A., Alonso, J., González, P., & del Coz, J. J. (2023). An equivalence analysis of binary quantification methods. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 6944–6952). (vol. 37).
- Esuli, A., Fabris, A., Moreo, A., & Sebastiani, F. (2023). Learning to Quantify. Cham, CH: Springer. <https://doi.org/10.1007/978-3-031-20467-8>
- Esuli, A., Moreo, A., & Sebastiani, F. (2018). A recurrent neural network for sentiment quantification. In *Acm international conference on information and knowledge management* (pp. 1775–1778). <https://doi.org/10.1145/3269206.3269287>
- Esuli, A., Moreo, A., Sebastiani, F., & Sperduti, G. (2022). A detailed overview of lequa@CLEF 2022: learning to quantify. In *CLEF 2022 - Conference and labs of the evaluation forum* (pp. 1849–1868). (vol. 3180). CEUR Workshop Proceedings.
- Forman, G. (2008). Quantifying counts and costs via classification. *Data Mining and Knowledge Discovery*, 17(2), 164–206. <https://doi.org/10.1007/s10618-008-0097-y>
- González, P., Castano, A., Chawla, N. V., & Coz, J. J. D. (2017). A review on quantification learning. *ACM Computing Surveys (CSUR)*, 50(5), 1–40.
- González, P., Castano, A., Peacock, E. E., Díez, J., Del Coz, J. J., & Sosik, H. M. (2019). Automatic plankton quantification using deep features. *Journal of Plankton Research*, 41(4), 449–463.
- González, P., Moreo, A., & Sebastiani, F. (2024). Binary quantification and dataset shift: An experimental investigation. *Data Mining and Knowledge Discovery*, (pp. 1–43).
- González-Castro, V., Alaiz-Rodríguez, R., & Alegre, E. (2013). Class distribution estimation based on the hellinger distance. *Information Sciences*, 218, 146–164. <https://doi.org/10.1016/j.ins.2012.05.028>
- Kornblith, S., Norouzi, M., Lee, H., & Hinton, G. (2019). Similarity of neural network representations revisited.
- Lezcano-Casado, M. (2019). Trivializations for gradient-based optimization on manifolds. In *Advances in neural information processing systems, neurIPS* (pp. 9154–9164).
- Lipton, Z., Wang, Y.-X., & Smola, A. (2018). Detecting and correcting for label shift with black box predictors. In *ICML* (pp. 3122–3130). PMLR.
- Maletzke, A., dos Reis, D., Cherman, E., & Batista, G. (2019). DyS: A framework for mixture models in quantification. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 4552–4560). (vol. 33).
- Moreno-Torres, J. G., Raeder, T., Alaiz-Rodríguez, R., Chawla, N. V., & Herrera, F. (2012). A unifying view on dataset shift in classification. *Pattern Recognition*, 45(1), 521–530. <https://doi.org/10.1016/j.patcog.2011.06.019>
- Moreo, A., Esuli, A., & Sebastiani, F. (2021). Quapy: A python-based framework for quantification. In *Proceedings of the 30th ACM international conference on information & knowledge management* (pp. 4534–4543).
- Pérez-Mon, O., Moreo, A., del Coz, J. J., & González, P. (2024). Quantification using permutation-invariant networks based on histograms. *Neural Computing and Applications*, <https://doi.org/10.1007/s00521-024-10721-1>
- Qi, L., Khaleel, M., Tavanapong, W., Sukul, A., & Peterson, D. (2021). A framework for deep quantification learning. In *Machine learning and knowledge discovery in databases: European conference, ECML PKDD 2020, ghent, belgium, september 14–18, 2020, proceedings, part i* (pp. 232–248). Springer.
- Saerens, M., Latinne, P., & Decaestecker, C. (2002). Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure. *Neural Computation*, 14(1), 21–41. <https://doi.org/10.1162/089976602753284446>
- Schumacher, T., Strohmaier, M., & Lemmerich, F. (2021). A comparative evaluation of quantification methods. *arXiv preprint arXiv:2103.03223*.
- Sebastiani, F. (2020). Evaluation measures for quantification: An axiomatic approach. *Information Retrieval Journal*, 23(3), 255–288. <https://doi.org/10.1007/s10791-019-09363-y>
- Shi, Y., & Jain, A. K. (2019). Probabilistic face embeddings. In *Proceedings of the IEEE/CVF international conference on computer vision (ICCV)*.

- Smith, N. A., & Tromble, R. W. (2004). Sampling uniformly from the unit simplex. *Johns Hopkins University, Tech. Rep*, 29.
- Sun, Y.-T., Huang, Y.-H., Ma, L., Lyu, X., Cao, Y.-P., & Qi, X. (2024). Splatter a video: Video gaussian representation for versatile processing. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, & C. Zhang (Eds.), *Advances in neural information processing systems* (pp. 50401–50425). Curran Associates, Inc. (vol. 37). <https://doi.org/10.52202/079017-1597>
- Yu, F. X., Choromanski, K., Kumar, S., Jebara, T., & Chang, S.-F. (2014). On learning from label proportions. *arXiv preprint arXiv:1402.5902*, .
- Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. R., & Smola, A. J. (2017). Deep sets. *Advances in Neural Information Processing Systems*, 30.